P3587R0
Reconsider reflection access for C++26

Published Proposal, 2025-01-13

Author:
Lauri Vasama

Audience:
EWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Current draft:
vasama.github.io/wg21/D3587.html

Current draft source:
github.com/vasama/wg21/blob/main/D3587.bs

Abstract

Reconsider access to arbitrary private data provided by P2996.

Table of Contents

§ 1. Introduction

The killer feature of reflection is automation. Automating the generation of boilerplate that could have already been written by hand. [P2996R8] goes well beyond that in allowing and effectively condoning access to private members of arbitrary types. The automation of boilerplate generation is fantastically well motivated. The same cannot be said for access control bypassing.

§ 1.1. Metadata versus data

It is important to make a clear distinction between access to private *metadata* and access to private *data*. Metadata includes things such as the size and alignment of a type, the types and offsets of its member variables, and the signatures of its member functions, et cetera. The data is the values stored

in the member variables of an object and the prvalues produced by taking the address of one of its member functions or static members. In a sense the metadata access already exists, only it can't yet be automated. The programmer can manually inspect the definition a type, even an external one to which they would not otherwise have access, and produce any manner of type trait specialisation manually. But without reflection and access to private *metadata* they cannot automate it.

```
// Used for some library optimisations when it is known that the type does
// contain any pointer members.
template<typename T>
inline constexpr bool contains_pointers = true;

// The definition of some_external_type was manually inspected to arrive at
// value. If you update the library version in the package manager depender
// remember to also check the definition again and update this if needed.
template<>
inline constexpr bool contains_pointers<some_external_type> = false;
```

All in all, there are a number of points in favour of access to private *metadata*:

1. It has been shown that custom type traits are useful.
2. The access is extended only to compile-time information, making it safe.
3. Users can already implement custom type traits manually in a fragile way.

For these reasons we are not opposed to access to arbitrary private *metadata*.

§ 1.2. Access to object data

Accessing private data is undoubtedly useful and we are not opposed to that. However there is a stark difference between accessing private data with permission and breaking through access controls to access *arbitrary* private data, e.g. that of `std::string`.

Access to private data is often presented as something necessary for useful reflection, but that is not precisely correct. Access to certain private data is indeed necessary, but access to *arbitrary* private

data is not. It is entirely possible to construct protocols for types to opt into automatic hashing or serialization. The simplest way to do that is by befriending:

```
class my_class {
    int private_data_to_serialize;

    // Give access to the serialization library:
    friend serialization_lib::access;
}
```

It's important to understand that automatic serialization of arbitrary types without guidance specific to that type (e.g. in the form of annotations) is not possible.

§ 2. Invariants

In general, private access control implies invariants. Access control is the mechanism provided by the language for *maintaining* those invariants. Only code within the class or within befriended entities may use private members, because only such code can be trusted to understand that invariants applied to those members.

It is not possible for a generic serialization or hashing or other algorithm to divine the arbitrary invariants applied to a private member. This includes more obvious invariants such as restrictions on the values of data members, but also more insidious ones such as being restricted to be accessed only by a specific thread.

When a member is being mutated by another thread, even reading it just for debug purposes is dangerous. Even seemingly simple types such as `std::string_view` or pointers to members, being larger than what most CPU architectures can copy in a single operation, can easily result in invalid values in the presence of data races. It's easy to see how a `std::string_view` with its address member read before a write and its size member after results in an unusable value that at best points to garbage and at worst to a memory-mapped register producing side-effects when read.

It might seem attractive to constrain algorithms based on the types of members, for example by rejecting any types containing pointers. This is entirely insufficient to alleviate the problem but even if it weren't, pointers can hide in seemingly safe types such as integers. And this is not as contrived as it might seem at first glance. Types such as tagged pointers are increasingly common in low-level or high-performance scenarios where it is important to save space. There is even a proposal to add language support for tagged pointers: [P3125R0].

The conclusion is that [P2996R8] proposes to break a very old and fundamental assumption of the language: that private members are safe from outside access. Not just safe from mutation but safe from data races.

3. Arguments in favour

Various pieces of motivation have been provided in favour of bypassing access control. In this section we go over all pieces of motivation known to us.

3.1. Serialization and hashing

Serialization and hashing have long been two poster child examples of reflection use cases. Both often involve repetitive listing of non-static data members. Whenever a new member is added, the developer must remember to add it to the list of things to hash or to serialize. This represents a perfect opportunity for automation using reflection. As explained in {#invariants} however, it is not possible for generic code to maintain arbitry invariants. Real serialization involving private members inevitably requires guidance. In many languages this comes in the form of annotations such as those proposed in [P3394R0]. It cannot be said that opting in is too inconvenient when in practice annotations or another mechanism to provide guidance is already necessary:

```
class my_class {
    int private_data_to_serialize;

    [[=serializer_lib::transient]]
    void* transient_pointer;

    friend serializer_lib::access;
};
```

3.1.1. P2996 Hashing example

[P2996R8] shows the following hashing use case, based on the [N3980] API:

```
template <typename H, typename T> requires std::is_standard_layout_v<T>
void hash_append(H& algo, T const& t) {
  template for (constexpr auto mem : nonstatic_data_members_of(^T)) {
```

```
        hash_append(algo, t.[:mem:]);
    }
}
```

After implementing a simple hash algorithm based on `std::hash` for types where available, we can see it in action:

```
my::hash_algo algo = {};
hash_append(algo, std::expected<int, const char*>(42));
std::print("hash = {}\n", algo.accumulator);
```

On Compiler Explorer: Clang

Here `std::expected`, which is equality comparable but not hashable out of the box, is made hashable through the use of the universal hashing algorithm. The problem with this is that `std::expected` contains a union and it is not possible for a generic algorithm to detect the active member of that union. Naively, the algorithm presented in [P2996R8] ignores the presence of the union and proceeds to visit each non-static data member regardless. Not only does this result in undefined behaviour, but the algorithm didn't even manage to produce a valid hash:

```
std::expected e1 = ...;
std::expected e2 = ...;

size_t h1 = hash(e1);
size_t h2 = hash(e2);

std::print("e1 == e2 : {}\n", (e1 == e2) ? "true" : "false");
std::print("h1 == h2 : {}\n", (h1 == h2) ? "true" : "false");
```

Execution of this program resulted in the following output:

```
e1 == e2 : true
h1 == h2 : false
```

On Compiler Explorer: Clang, Screenshot on GitHub

This is not to say that an expert C++ programmer could not come up with rules to constrain the set of types to which such an algorithm can be applied, e.g. by rejecting any type containing unions, among others. Although it must be pointed out that no amount of expertise can protect the algorithm from

data races arising due to usually safe mutation of private data by another thread. This example does showcase how the sort of implementations that might be created by regular developers (e.g. by copying code from a WG21 paper) present a massive footgun.

After making the generic hashing algorithm more and more conservative, one finds that the safe end result is an algorithm constrained to work only on the set of class types containing no private bases or non-static data members, thus obviating the proposed motivation for accessing arbitrary private *data*. Note that at the same time, it is useful to determine whether a type has any private members in order to reject it, once again reinforcing the need for access to private *metadata*.

3.2. Existing codebases

It has been argued that users with large codebases may wish to apply reflection to a large number of existing types automatically, say for the purposes of serialization, and that having to provide access to the reflecting code (e.g. by befriending) is *cumbersome*. There are a few flaws with this reasoning:

1. It is not possible to correctly serialize arbitrary types with invariants.
2. Any such existing code very likely already includes manually written equivalents of the code soon to be generated through reflection. In the switch to automatic serialization, that code likely has to be removed.

For these reasons, any such types would in any case have to undergo manual audits and modifications before a generic reflection based serializer can be applied. It is hard to imagine a scenario where reflection could be succesfully applied to a large number of existing types with invariants for any non-trivial use case and without any changes to the types in question.

3.3. External code

3.3.1. Third party libraries

Some users have expressed a desire to access the internals of third-party libraries in order to implement features not provided in the public API of the library. This may involve reading or writing private member variables or invoking private member functions. Proponents will argue that cooperating with library authors and upstreaming changes is difficult; it requires getting along with people. Forking and maintaining customized versions of libraries for private use is considered to be too much effort.

The standard library itself is often used as an example of this. Despite the introduction of `resize_for_overwrite`, some users would like to resize standard library containers without

initializing their elements. There are good reasons for wanting to do this - particularly asynchronous initialization - but it is a problem to be solved in the committee; a social problem. (Or alternatively as a non-standard extension.)

3.3.2. Legacy code

Some users depend on libraries that simply cannot ever be changed, for whatever political reasons. This could be considered yet another social problem, but for the sake of the argument we shall consider it a business requirement. This begs the question of why whatever entity is blocking changes to this code would instead allow accessing its internals and potentially violating its invariants. Perhaps it is thought that an access bypassing solution applied from the outside might fly under the radar if not too loudly advertised.

Using reflection to automate certain tasks such as serialization has the benefit of making the code more resistant to changes. It is not possible to forget to add a member variable to the list of those to be serialized when it is done automatically. However, when the premise is that a library can never change, there is no such benefit gained.

3.3.3. Non-generic access

These examples provide some motivation for breaking through controls to access *specific* members which invariants and usage is *understood* by the developer. Using the facilities proposed in [P2996R8], this would likely be achieved using something like the following accessor:

```
consteval std::meta::info get_nsdm_helper(
  std::meta::info type,
  std::string_view name) {
  for (auto nsdm : nonstatic_data_members_of(type)) {
    if (has_identifier(nsdm) && identifier_of(nsdm) == name)
      return nsdm;
  }
  std::unreachable();
}

template<static_string Name, typename T>
decltype(auto) get_nsdm(T&& object) {
  return std::forward_like<T>(object.[:get_nsdm_helper(^^T, Name):])
}
```

```
template<typename T>
void resize_vector_uninitialized(std::vector<T>& vec, size_t new_size) {
    vec.reserve(new_size);

#if defined(_MSVC_STL_VERSION)
    auto& pointers = get_nsdm<"_Mypair">(object)._Myval2;
    pointers._Mylast = pointers._Myfirst + new_size;
#elif ...
    // Handle other implementations...
#endif
}
```

This seems to have very little to do with *reflection itself*, which is only used here as a tool for breaking through access controls. There is no automation of any task or decisions made based on reflecting the properties of a type. In fact this sort of use case would be much better served by a different kind of facility providing direct access to private members:

```
template<typename T>
void resize_vector_uninitialized(std::vector<T>& vec, size_t new_size) {
    vec.reserve(new_size);

#if defined(_MSVC_STL_VERSION)
    auto& pointers = object.private _Myval2;
    pointers._Mylast = pointers._Myfirst + new_size;
#elif ...
    // Handle other implementations...
#endif
}
```

Here the strawman syntax `x.private y` is used for accessing the private member `y` of object `x`.

These sorts of examples have been presented in the past as rebuttals to questions about the necessity for access to arbitrary private data, but to our knowledge have not been presented in any papers targeting EWG. Perhaps these are valuable use cases, but in that case it should be brought to the committee in a separate paper and considered on its own merits.

Note that, hypothetically, if the above strawman syntax were added to the language, it would be very easy to search for and it would automatically benefit from reflection, as other existing language constructs do: `x.private [: info :]`. Note also that this paper does not propose the addition of any new syntax.

3.4. Debugging: universal formatter

[P2996R8] provides only a single motivating example for debugging: the universal formatter. While dumping class fields purely for debugging purposes is undoubtedly a useful thing to do, there are a few important things to note:

1. This is purely a debugging facility. It is not possible to correctly deserialize the output for arbitrary types.
2. When accessing the private data members of arbitrary types, normally safe concurrent modifications internal to the class cannot be ruled out. In this scenario the dumping code may very easily invoke undefined behaviour through data races. These races may be benign if the dumping code is careful to recurse through subobjects and only print out scalars. The worst case then would be sanitizer warnings or tearing depending on the platform and data type.

   However, if the dumping code does anything more complicated, such as printing some values using `std::format`, those data races will very quickly turn malignant. Dereferencing a torn or stale pointer will clearly result in unpredictable consequences. The best case scenario is a crash or reading garbage. The worst might be an observable side effect if a read through an invalid pointer happened to hit a memory mapped register or a guard page. The universal formatter example shown in [P2996R8] falls into this trap.

3.5. The language is already broken

3.5.1. The pointer-to-member NTTP trick

Proponents of access bypassing argue that because the language already allows bypassing access controls through an obscure and convoluted trick involving explicit class template specializations with a pointer-to-member NTTP, there is no further harm in allowing easy access to all privates:

```
class Victim { int secret = 42; };

int& rob_secret(Victim& v);

template <int Victim::* M>
struct Robber {
    friend int& rob_secret(Victim& v) {
        return v.*M;
    }
};
```

```
template struct Robber<&Victim::secret>;

void f(Victim& v) {
    std::print("v.secret = {}\n", rob_secret(victim));
}
```

See it on Compiler Explorer

1. This trick is extremely obscure and not widely used.
2. Applying it requires defining individual explicit template instantiations for each member being accessed.
3. It comes with a host of platform specific limitations. See here: GitHub

Proponents also claim that without first-class access control bypassing, users will simply apply reflection to automate access using this trick. While some level of automation might be achievable using token injection, there is certainly no way of automating this using the facilities proposed in [P2996R8].

3.5.2. Accessing object representations

Similarly it is argued that given access to arbitrary private metadata, including non-static data member offsets, it is also possible to access the data using `reinterpret_cast` and pointer arithmetic, and that it is better to allow direct access than to risk users implementing the same thing in unscrupulous ways:

```
class privates {
    std::string s;

public:
    privates()
        : s("secret") {}
};

void f(privates& p) {
    template for (constexpr auto member : nonstatic_data_members_of(^^priva
        void* m = reinterpret_cast<unsigned char*>(&p) + offset_of(member).
```

```
            std::print("{}", *static_cast<typename [: type_of(member) :]*>(m));
        }
    }
```

On Compiler Explorer: Clang

This code has undefined behaviour and thanks to the efforts of educators in our community, more than ever, our users understand that. This sort of code sticks out like a sore thumb in code review. In any case, much like the member pointer specialisation trick, this "solution" only provides access to a subset of private members, while still leaving others protected. This is to say that the proposed changes in [P2996R8] do inevitably open up access to previously inaccessible members.

§ 4. Library authors

For library authors, access to arbitrary private data represents a colossal increase in API surface area. With the condonement by the language to access private members, any visible private member of a class is implicitly made part of a library API. The ABI surface areas of libraries may also be increased, though not to the same extent because many private members - particularly private non-static data members - already contribute to library ABIs.

Even when library authors choose to ignore API breaks caused only by private members, the inevitable complaints about them will create an undue burden on the maintainer. To combat these issues, we may see increased use of the PIMPL pattern (cppreference.com) as a defensive measure, potentially introducing runtime inefficiency and preventing optimisations. Additionally we may see calls for a way to once more prevent some members from being reflected, e.g using a new access control syntax (strawman syntax):

```
class future_class {
true private:
  int non_reflectable_member;
};
```

§ 5. Implications for memory safety

[P3390R0] proposes a set of extensions for authoring provably memory-safe code in C++. In part this relies on the ability to author safe class types (e.g. a safe alternative to `std::string`) containing unsafe code in their implementations. In this scenario the uncheckable invariants applied to private

members become even more significant than before. Unregulated outside access to the member data of such a type cannot be allowed in any safe context. This requirement encompasses members of a fundamentally safe type such as integer types as well as read-only access due to the potential for data races.

Even in the absence of provable memory safety, access to arbitrary private member data must be considered an unsafe operation. [P3081R0] proposes a set of safety profiles to mitigate existing safety problems. If C++26 ships access breaking reflection, should it also ship an `invariant_safety` profile rejecting access to private data?

6. What to do instead?

The crux of this proposal is that C++26 should not provide access to *arbitrary private data*. Access to arbitrary private *metadata* and access to private data when given permission are both acceptable and the latter is extremely important for useful reflection.

This paper proposes to ship a minimum viable reflection facility in C++26 without the unsafe and unnecessary access to arbitrary private data. One that provides ways to automate the tasks already performed by developers without fundamentally changing the language.

If C++26 ships with access breaking, it is unlikely that the committee can ever remove it.
If C++26 ships without access breaking, it is trivial to introduce it in C++29 after further consideration.

Even without access breaking [P2996R8] will make a fantastic addition C++ and undoubtedly bring great advancements in developer productivity.

6.1. Options

6.1.1. Splicing Should Respect Access Control

[P3473R0] proposes applying usual access checks to splice expressions:

```
class S {
  int priv;
public:
  S() : priv(0) {}
};
```

```
consteval auto get_member(std::meta::info info, std::string_view name) {
  for (std::meta::info field : nonstatic_data_members_of(info)) {
    if (has_identifier(field) && identifier_of(field) == name)
      return field;
  }
  std::unreachable();
}

int main() {
    S s;

    // Accessing private metadata is still fine:
    constexpr std::meta::info S_p = get_member(^^S, "p");

    // Accessing private data without permission is rejected:
    int s_p = s.[: S_p :];
}
```

This is the authors' preferred solution. It prevents the problematic data access, while permitting the useful metadata access. The changes to standard are minimal. If the calculus for private data access changes in the C++29 timeframe, it is easy to lift this restriction.

6.1.2. `access_context::unchecked()`

[P3547R0] proposes the addition of `std::meta::access_context` to describe the permissions for accessing members of types. Included in that paper is `access_context::unchecked()` to produce an access context with unrestricted access to all members.

Accepting [P3547R0] but removing `access_context::unchecked()` with no replacement also meets the primary goal of this paper, but unfortunately prevents access to private metadata. The changes to the standard are minimal.

6.2. Useful reflection without unrestricted access

If access breaking is removed from C++26, the inevitable question is how to achieve all those common tasks involving private members that reflection is supposed to solve.

Using the existing C++ rules, users could simply opt-in using friend declarations.

6.2.1. Access-aware splicing

If option 1. along with [P3547R0] (or something like it) is accepted, in the future the splicing syntax could be extended to take an optional access context object along with the `meta::info`:

```
// Splice using access of the specified context:
x.[: private_member_info, context :]
```

6.2.2. Metaclass decorators

This example uses a Python-like syntax for decorators that could be introduced in the future and used to transform class definitions in a way similar to the metaclass proposal [P0707R5].

```
@serializer_lib::serializable
class my_class {
    int private_data_to_serialize;

    [[=serializer_lib::transient]]
    void* transient_pointer = nullptr;

public:
    // Public members...
};
```

The `@serializer_lib::serializable` decorator could be used to automatically transform the class definition in the following way, among other possibilities:

```
class my_class {
    int private_data_to_serialize;

    [[=serializer_lib::transient]]
    void* transient_pointer = nullptr;

public:
    // Public members...

private:
    // Ensure that the library is able to construct the object.
    my_class(serializer_lib::deserializer& d)
```

```
        : private_data_to_serialize(d.deserialize<int>())
    {}

    // Ensure that the library is able to access private members.
    friend serializer_lib::access;
};
```

Note that in this case thanks to the generated code, befriending the serializer is not strictly necessary. Alternative protocols could be established in order to allow the serializer library to construct the class during deserialisation.

This closely resembles Rust's much praised serialization framework Serde:

```
#[derive(Serialize, Deserialize, Debug)]
struct Point {
    x: i32,
    y: i32,
}
```

§ References

§ Informative References

[N3980]
H. Hinnant, V. Falco, J. Byteway. *Types don't know #*. 24 May 2014. URL: https://wg21.link/n3980

[P0707R5]
Herb Sutter. *Metaclass functions for generative C++*. 16 October 2024. URL: https://wg21.link/p0707r5

[P2996R8]
Barry Revzin, Wyatt Childers, Peter Dimov, Andrew Sutton, Faisal Vali, Daveed Vandevoorde, Dan Katz. *Reflection for C++26*. 17 December 2024. URL: https://wg21.link/p2996r8

[P3081R0]
Herb Sutter. *Core safety Profiles: Specification, adoptability, and impact*. 16 October 2024. URL: https://wg21.link/p3081r0

[P3125R0]
Hana Dusíková. *Pointer tagging*. 22 May 2024. URL: https://wg21.link/p3125r0

[P3390R0]

Sean Baxter, Christian Mazakas. *Safe C++*. 12 September 2024. URL: https://wg21.link/p3390r0

[P3394R0]

Daveed Vandevoorde, Wyatt Childers, Dan Katz,. *Annotations for Reflection*. 14 October 2024. URL: https://wg21.link/p3394r0

[P3473R0]

Steve Downey. *Splicing Should Respect Access Control*. 16 October 2024. URL: https://wg21.link/p3473r0